
Table of Contents

.....	1
1. Paramètres de simulation	1
2. Génération du signal d'entrée triangulaire	1
3. Génération du filtre gaussien avec $N_h = 30$	2
4. Simulation du problème direct: convolution centrée avec le filtre gaussien	3
5. Ajout du bruit de quantification	4
6. Détermination du bruit de quantification	5
7. Déterminer la dimension du vecteur de sortie y_{nb}	6
8. Construction de la matrice de convolution H	6
9. Calcul du signal de sortie en approche matricielle	6
11. Déconvolution naïve sans bruit	8
12. Déconvolution naïve avec bruit	9
13. Influence de σ	10
14. Déconvolution par moindres carrés	10
18. Construction de la matrice de régularisation $D1$ (Méthode de l'aide fournie)	12
19. Résolution du problème régularisé pour plusieurs valeurs de λ	12
20. Détermination de la meilleure valeur de λ	13

```
clc; clear; close all;
```

1. Paramètres de simulation

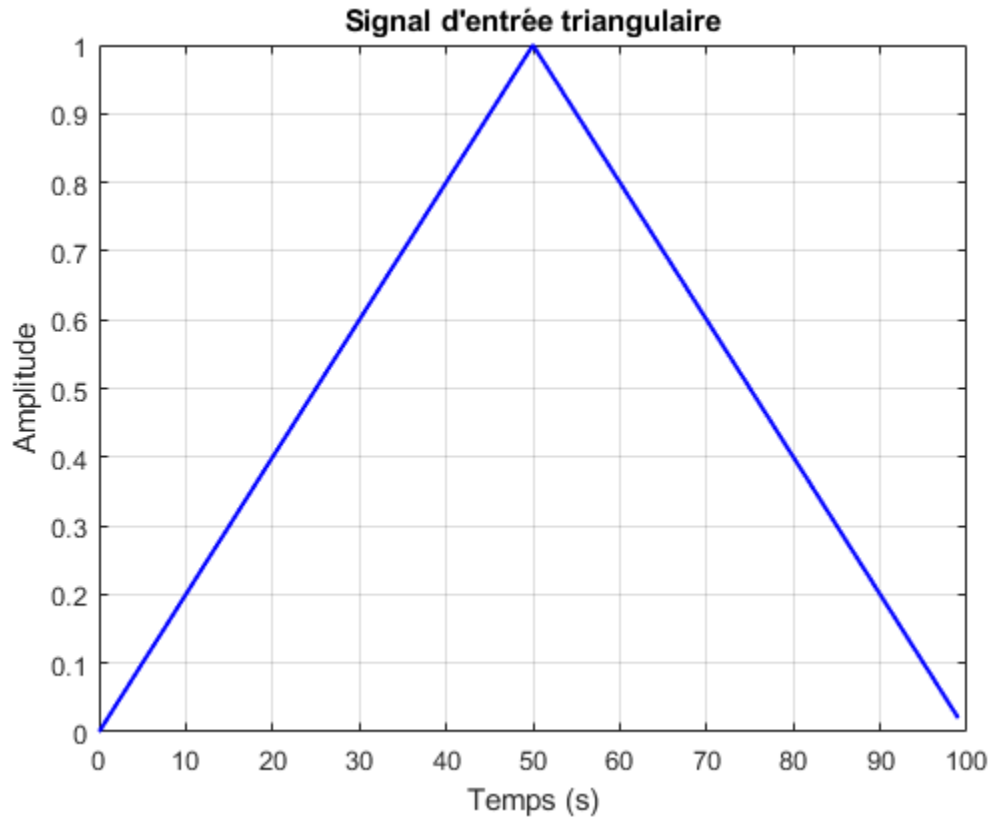
```
L = 100;      % Durée d'observation (s)
fe = 1;       % Fréquence d'échantillonnage (Hz)
Te = 1/fe;    % Période d'échantillonnage (s)
Nx = L/Te;    % Nombre d'échantillons
n = 0:Nx-1;   % Indices temporels
t = n*Te;     % Base de temps
```

2. Génération du signal d'entrée triangulaire

```
L_half = L/2;
x1 = (t(t <= L_half) / L_half);      % Première partie du triangle
x2 = ((L - t(t > L_half)) / L_half);  % Deuxième partie
x = [x1, x2];                         % Signal triangulaire complet
```

```
figure;
plot(t, x, 'b', 'LineWidth', 1.5);
title('Signal d\'entrée triangulaire');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```

```
% Comme souhaité on obtient un signal triangulaire de 100s
```



3. Génération du filtre gaussien avec $N_h = 30$

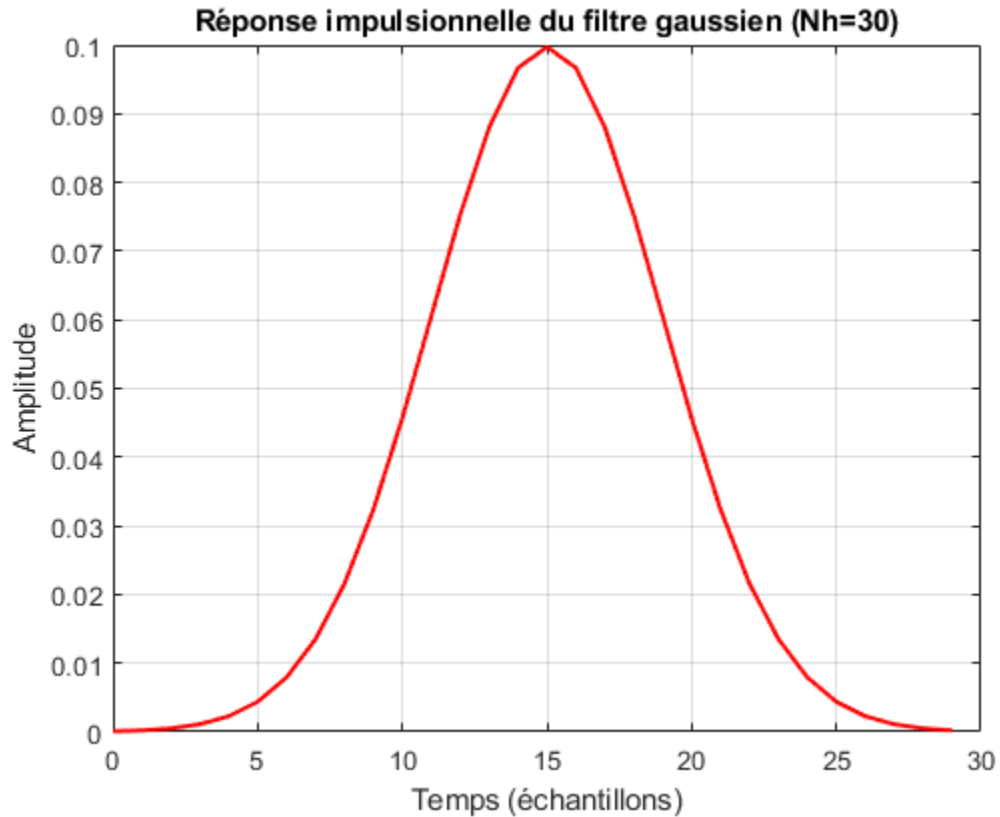
```
Nh = 30;          % Taille de la réponse impulsionnelle
n_h = 0:Nh-1;    % Indices temporels pour le filtre

m = 15;          % Moyenne de la gaussienne
sigma = 4;       % Écart-type de la gaussienne

% Calcul de la réponse impulsionnelle discrète
h = (1 / (sigma * sqrt(2*pi))) * exp(-((n_h - m).^2) / (2 * sigma^2));

% Affichage du filtre gaussien tronqué
figure;
plot(n_h, h, 'r', 'LineWidth', 1.5);
title('Réponse impulsionnelle du filtre gaussien (Nh=30)');
xlabel('Temps (échantillons)');
ylabel('Amplitude');
grid on;

% Le filtre gaussien de variance 4 et de moyenne 15 est obtenu sur 30s
```



4. Simulation du problème direct: convolution centrée avec le filtre gaussien

```
% Calcul de la convolution
```

```
ynb = conv(x, h, 'same');
```

```
% Affichage du signal de sortie non bruité
```

```
figure;
```

```
plot(t, ynb, 'g', 'LineWidth', 1.5);
```

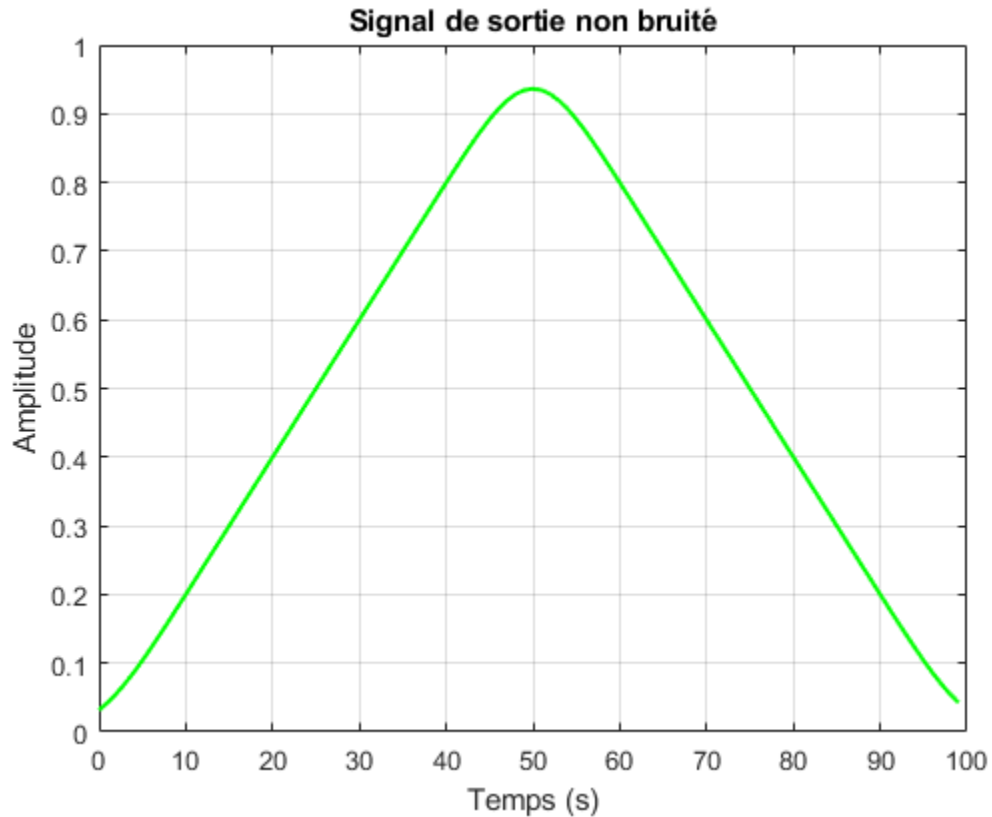
```
title('Signal de sortie non bruité');
```

```
xlabel('Temps (s)');
```

```
ylabel('Amplitude');
```

```
grid on;
```

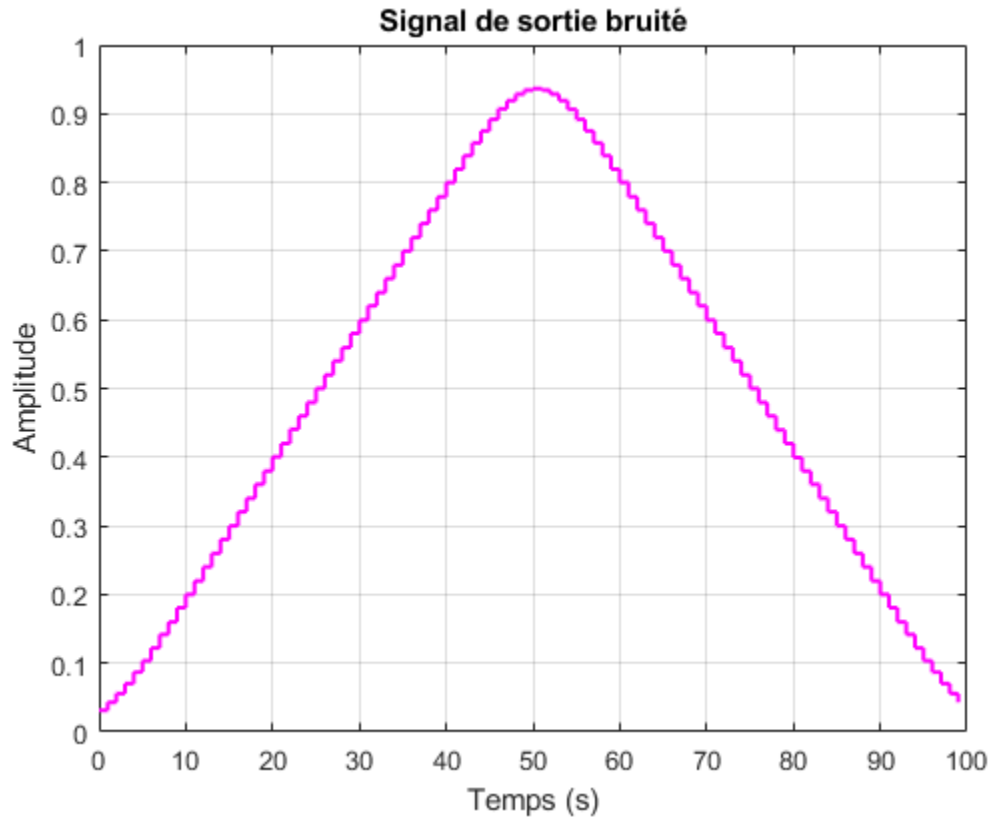
```
% Obtention du signal convolué avec le filtre, same pour être de longueur  
%100
```



5. Ajout du bruit de quantification

```
% Simulation du convertisseur ADC avec quantification sur 10 bits  
y = round(ynb * 2^10) / 2^10;
```

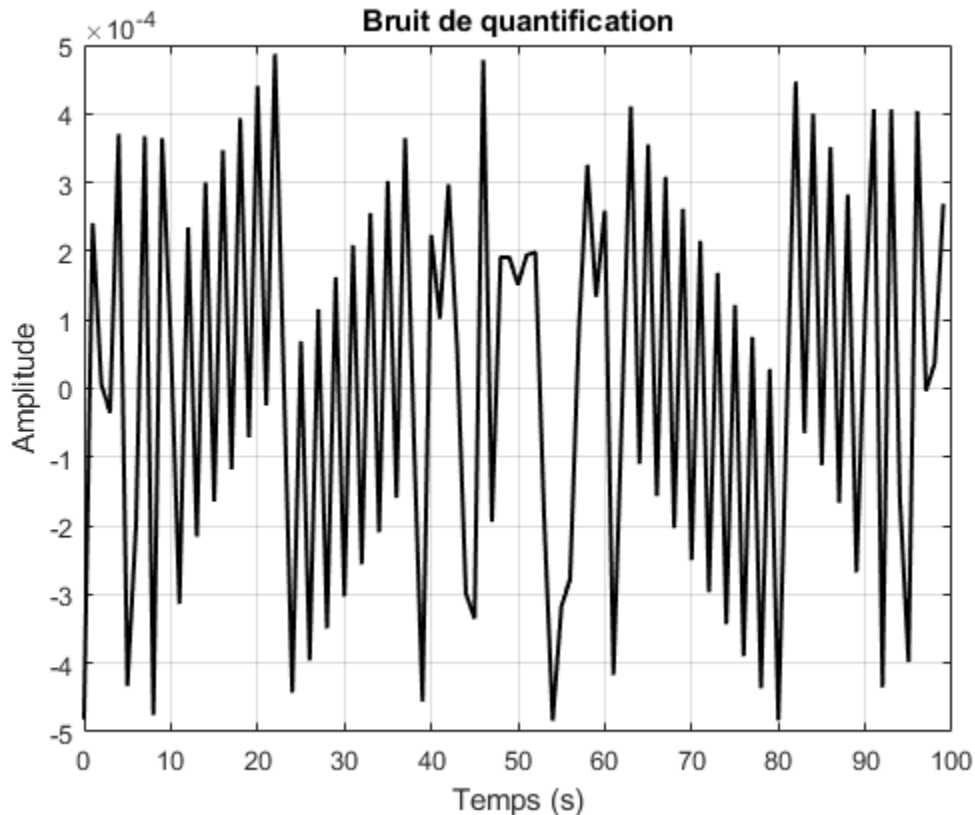
```
% Affichage du signal quantifié  
figure;  
stairs(t, y, 'm', 'LineWidth', 1.5);  
title('Signal de sortie bruité');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;
```



6. Détermination du bruit de quantification

```
% Calcul du bruit de quantification
b = y - ynb;

% Affichage du bruit de quantification
figure;
plot(t, b, 'k', 'LineWidth', 1.5);
title('Bruit de quantification');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```



7. Déterminer la dimension du vecteur de sortie y_{nb}

```
Ny = length(x); % On garde la même dimension que l'entrée x
```

8. Construction de la matrice de convolution H

```
% Création du premier vecteur colonne de H
col_H = [h(:); zeros(Ny - Nh, 1)];

% Création du premier vecteur ligne de H (les zéros sauf le premier élément)
row_H = [h(1), zeros(1, Ny-1)];

% Construction de la matrice de convolution avec toeplitz
H = toeplitz(col_H, row_H);
```

9. Calcul du signal de sortie en approche matricielle

```
ynb_matrix = H * x';
```

```

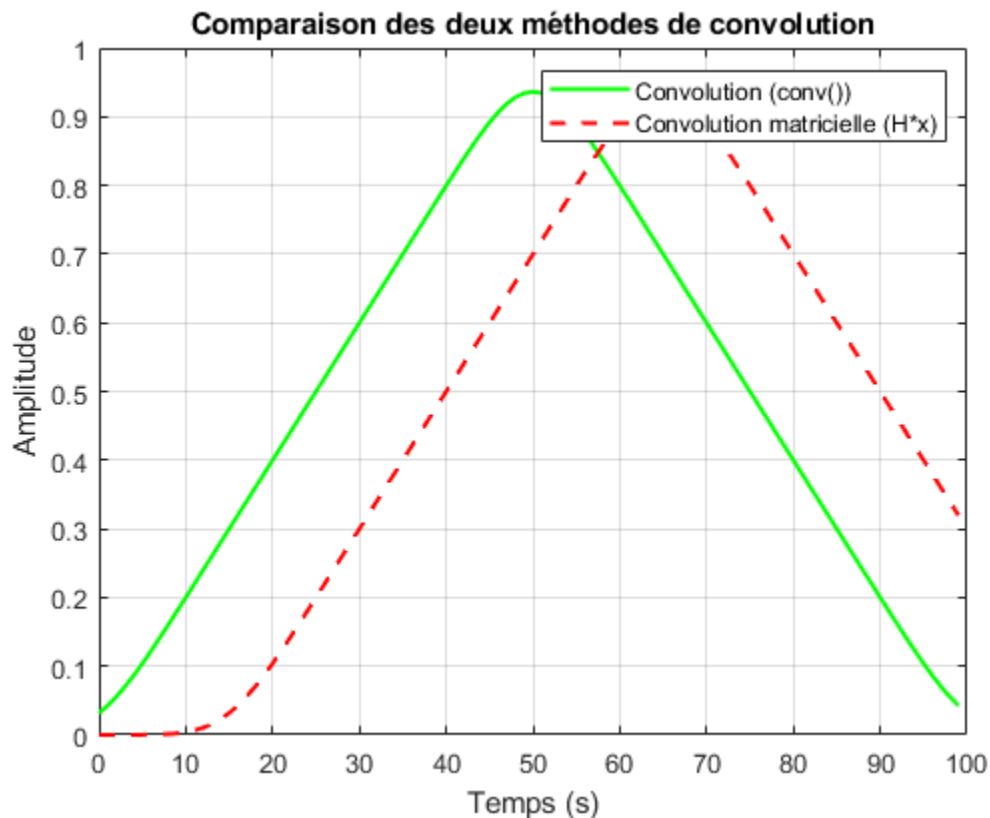
figure;
plot(t, ynb, 'g', 'LineWidth', 1.5); hold on;
plot(t, ynb_matrix, '--r', 'LineWidth', 1.5);
legend('Convolution (conv())', 'Convolution matricielle (H*x)');
title('Comparaison des deux méthodes de convolution');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

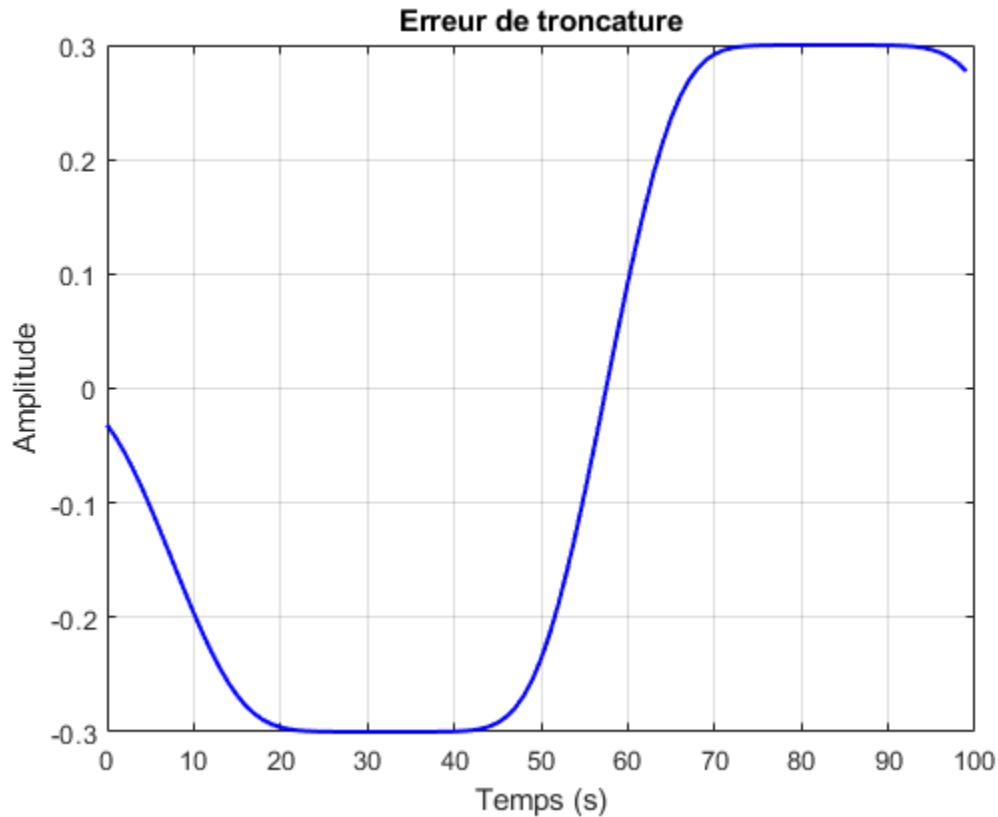
% On compare les deux graphes, il résulte un décalage pour le signal
% construit matriciellement du à la construction de la matrice toeplitz

% Calcul de l'erreur du à la troncature du signal obtenu par approche
% matricielle
erreur_troncature = ynb_matrix' - ynb;

% Affichage de l'erreur de troncature
figure;
plot(t, erreur_troncature, 'b', 'LineWidth', 1.5);
title('Erreur de troncature');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

```



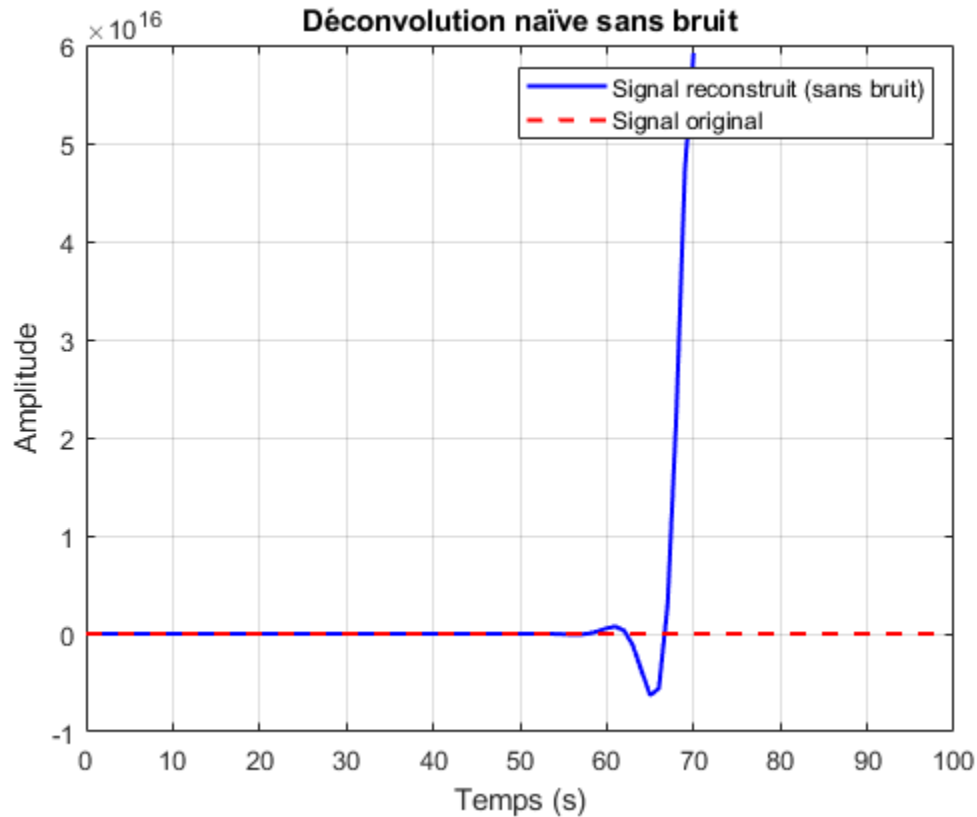


11. Déconvolution naïve sans bruit

```
% Déconvolution de y_nb pour retrouver x
[x_rec_nb, r_nb] = deconv(y_nb, h);

% Affichage du signal reconstruit sans bruit
figure;
plot(t(1:length(x_rec_nb)), x_rec_nb, 'b', 'LineWidth', 1.5);
hold on;
plot(t, x, '--r', 'LineWidth', 1.5);
legend('Signal reconstruit (sans bruit)', 'Signal original');
title('Déconvolution naïve sans bruit');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

%L'instabilité de l'opération de déconvolution provoque un signal
%complètement différents
```

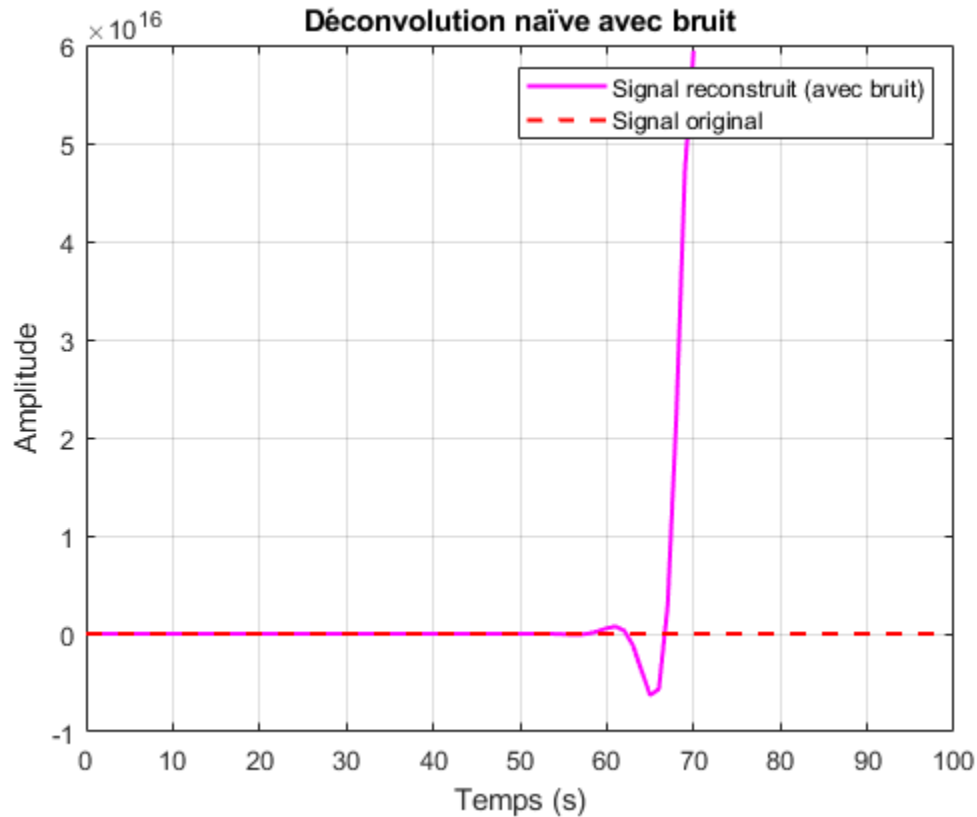



12. Déconvolution naïve avec bruit

```
[x_rec_bruit, r_bruit] = deconv(y, h); % Déconvolution du signal bruité

% Affichage du signal reconstruit avec bruit
figure;
plot(t(1:length(x_rec_bruit)), x_rec_bruit, 'm', 'LineWidth', 1.5);
hold on;
plot(t, x, '--r', 'LineWidth', 1.5);
legend('Signal reconstruit (avec bruit)', 'Signal original');
title('Déconvolution naïve avec bruit');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

% Le bruit ne change pas l'instabilité de la déconvolution
```



13. Influence de sigma

% On constate que quand sigma augmente la reconstruction est de plus en plus instable

14. Déconvolution par moindres carrés

Calcul de la pseudo-inverse de H solution des moindres carrés

```
H_pinv = pinv(H);

% Reconstruction du signal par moindres carrés
x_rec_ls = H_pinv * ynb';

% Affichage du signal reconstruit
figure;
plot(t, x_rec_ls, 'b', 'LineWidth', 1.5);
hold on;
plot(t, x, '--r', 'LineWidth', 1.5);
legend('Signal reconstruit (moindres carrés)', 'Signal original');
title('Déconvolution par moindres carrés (sans régularisation)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
```

```

% Calcul de l'erreur de reconstruction
erreur_reconstruction = x - x_rec_ls;

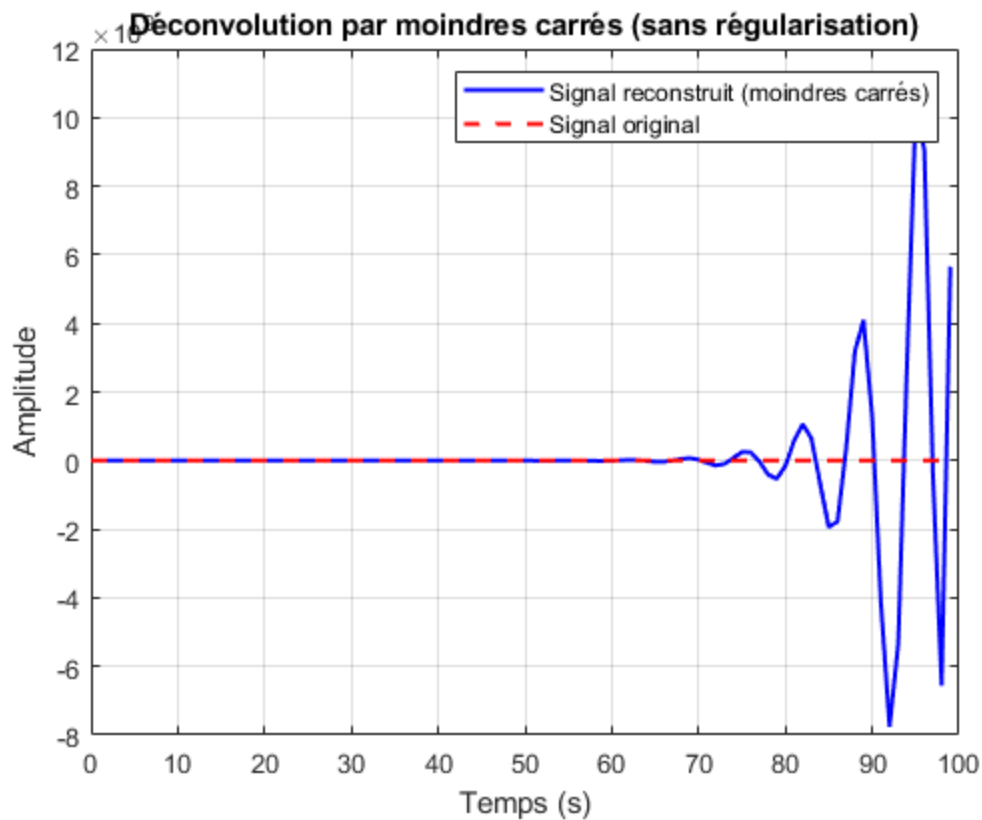
% Tracé de l'erreur de reconstruction
figure;
plot(t, erreur_reconstruction, 'k', 'LineWidth', 1.5);
title('Erreur de reconstruction (moindres carrés)');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

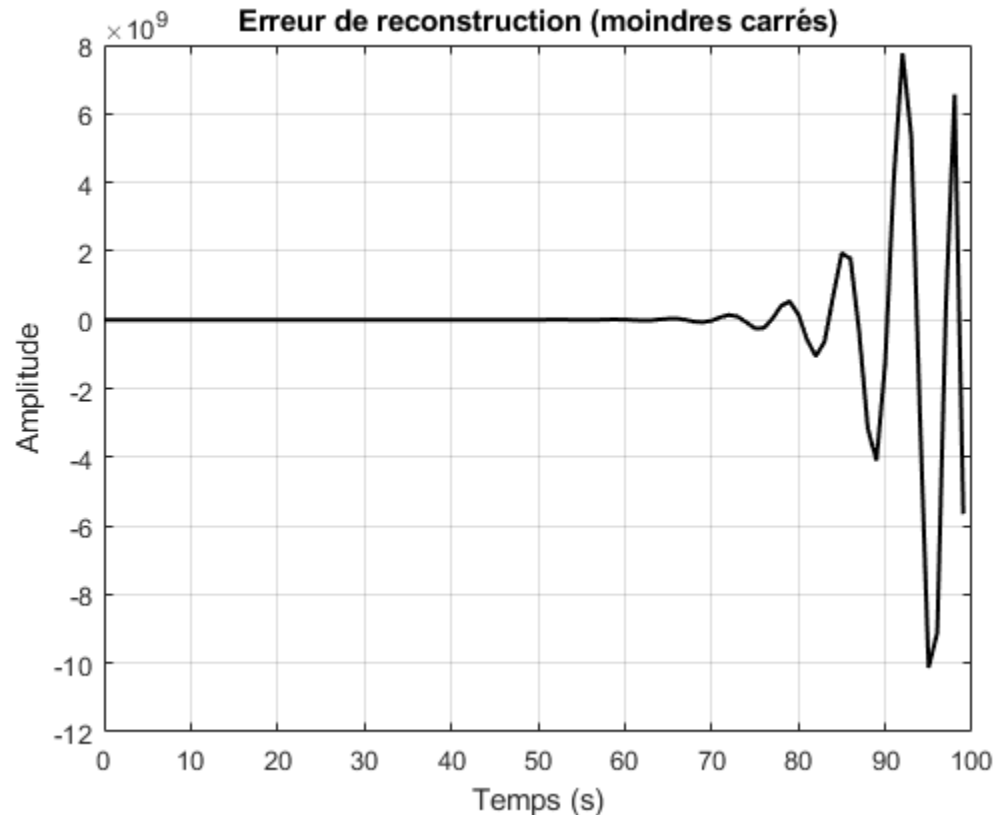
% Calcul de l'énergie de l'erreur de reconstruction
energie_erreur = norm(erreur_reconstruction)^2;
disp(['Énergie de l''erreur de reconstruction (moindres carrés) : ',
num2str(energie_erreur)]);

% L'énergie de l'erreur est très grande montrant ainsi que la
% reconstruction n'est pas fidèle, cela peut provenir de la méthode utilisé
% ou plus probablement de l'algorithme qui peut être faux.

Énergie de l'erreur de reconstruction (moindres carrés) :
4.125589071380073e+22

```





18. Construction de la matrice de régularisation D1 (Méthode de l'aide fournie)

```
dcol1 = zeros(Nx, 1);
dcol1(1:2) = [1; -1];
dlig1 = zeros(1, Nx);
dlig1(1) = 1;
D1 = toeplitz(dcol1, dlig1);
```

%Matrice de régulation construite comme proposé dans le sujet

19. Résolution du problème régularisé pour plusieurs valeurs de lambda

```
lambda_values = logspace(-7, 2, 10); % Valeurs de lambda testées
errors = zeros(length(lambda_values), 1); % Stockage de l'erreur

for i = 1:length(lambda_values)
    lambda = lambda_values(i);

    % Résolution du problème régularisé
    x_rec_reg = (H'*H+lambda*D1'*D1)\(H' * ynb');
```

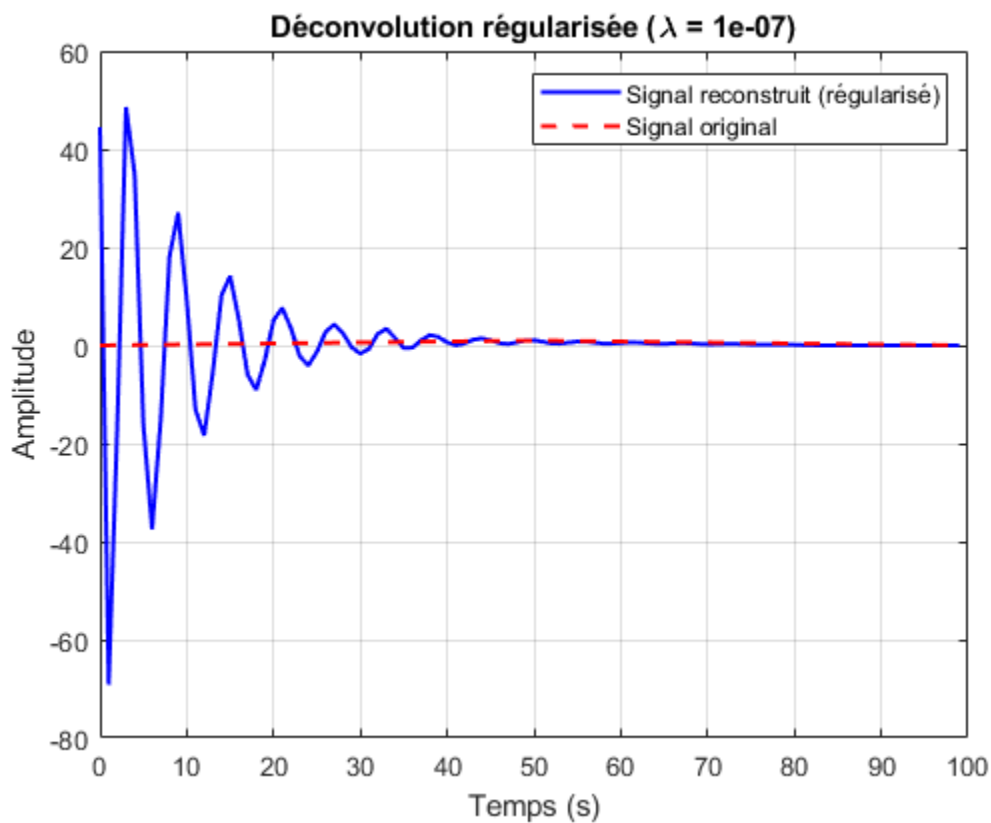
```

% Calcul de l'erreur de reconstruction
errors(i) = norm(x - x_rec_reg)^2;

% Affichage du premier signal reconstruit pour contrôle
if i == 1
    figure;
    plot(t, x_rec_reg, 'b', 'LineWidth', 1.5);
    hold on;
    plot(t, x, '--r', 'LineWidth', 1.5);
    legend('Signal reconstruit (régularisé)', 'Signal original');
    title(['Déconvolution régularisée (\lambda = ', num2str(lambda),
    ')']);
    xlabel('Temps (s)');
    ylabel('Amplitude');
    grid on;
end
end

%Le signal obtenu n'est toujours pas le même, ce qui ne semble pas normal.

```



20. Détermination de la meilleure valeur de lambda

```

% Recherche de la valeur de lambda qui minimise l'erreur de reconstruction
[~, idx_best_lambda] = min(errors);

```

```

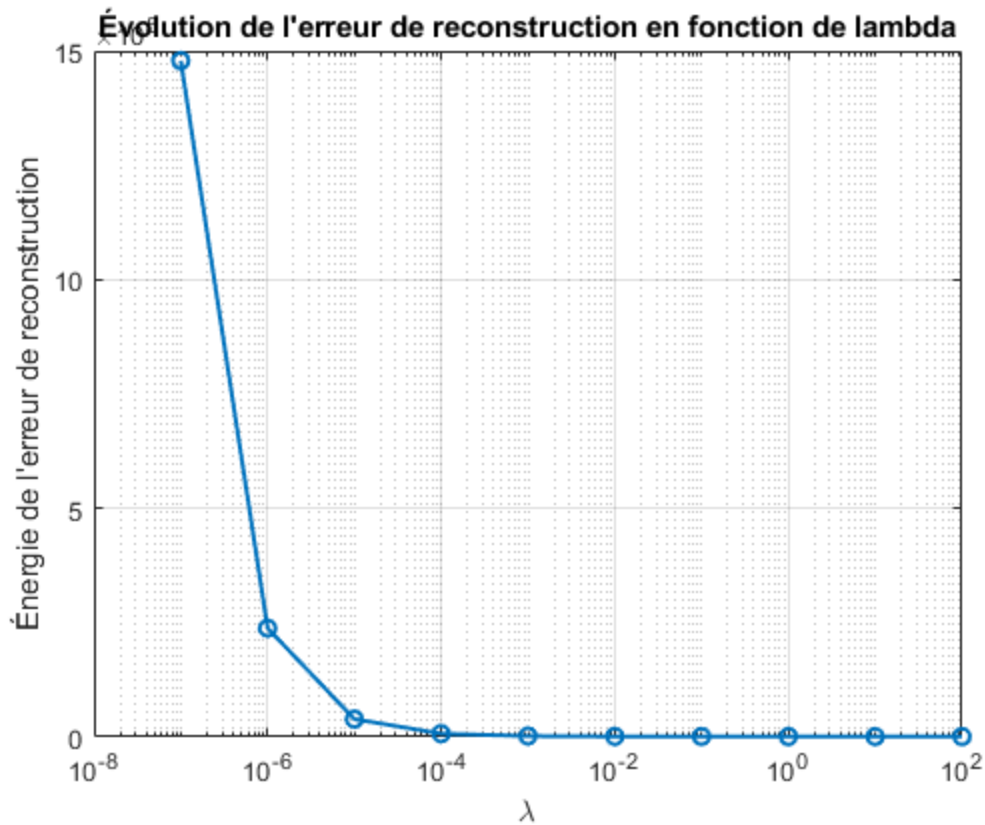
best_lambda = lambda_values(idx_best_lambda);

% Affichage de l'évolution de l'erreur en fonction de lambda
figure;
semilogx(lambda_values, errors, 'o-', 'LineWidth', 1.5);
title('Évolution de l''erreur de reconstruction en fonction de lambda');
xlabel('\lambda');
ylabel('Énergie de l''erreur de reconstruction');
grid on;

disp(['Meilleure valeur de lambda trouvée : ', num2str(best_lambda)]);

Meilleure valeur de lambda trouvée : 100

```



Published with MATLAB® R2024a